

面向 LLM / Agent Systems 科研的深度学习零基础课

第 03 讲

向量、Dot Product 与 “相似度”

本讲目标：真正理解向量点积为什么是“对应维度相乘再求和”，它为什么能表达匹配关系、又为什么不能简单等同于相似度，并把这一直觉连接到 Attention 的 $Q \cdot K$ 。

讲义完整版

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

目录

1 为什么这一讲是 Attention 的数学地基	3
1.1 本讲学完的最低验收标准	3
2 向量到底是什么？	3
2.1 【原理】向量首先是一串有顺序的数字	4
2.2 【通俗解释】向量就是“一条对象的数字身份证”	4
3 Dot Product：对应位置相乘，再求和	4
3.1 【必要公式】	5
3.2 【公式逐项拆解】	5
4 【具体数字例子】完整手算一次	5
5 为什么点积可以反映“方向关系”？	6
5.1 从最简单的二维向量开始	6
5.2 更完整的几何公式	7
6 “点积大 = 更相似”为什么只是近似说法？	7
7 Cosine Similarity：把长度影响除掉	7
7.1 【必要公式】	7
7.2 为什么这样做？	8
8 为什么神经网络喜欢点积？	8
8.1 原因 1：它把两个向量压缩成一个匹配分数	8
8.2 原因 2：计算极其规则，适合矩阵化和 GPU	8
8.3 原因 3：它可微分	8
9 从一个 Query 到多个 Key：第一次接近 Attention	8
10 为什么 Attention 还要除以 $\sqrt{d_k}$？先埋一个伏笔	9
11 【代码实验】先手工逻辑，再 NumPy	9
11.1 向量长度与余弦相似度	10
12 Shape 视角：不要只盯着数字	10

13 常见误区	11
14 科研视角：以后论文中你会怎样遇到它？	11
15 把本讲重新接回你已经学过的 LLM 概念	12
16 本讲知识链	12
17 本讲自检	12
18 下一讲预告：Matrix Multiplication	13

1 为什么这一讲是 Attention 的数学地基

前两讲解决了两个问题：第一，深度学习里的数据通常被组织成 Tensor；第二，Tensor 的 Shape 决定了数字怎样被组织和读取。

这一讲开始，我们问一个更关键的问题：**两个向量之间，怎样计算一个“匹配程度”的数字？**

Transformer 的 Attention 会反复做类似计算：

$$q \cdot k$$

其中 q 是 Query 向量， k 是 Key 向量。这个点积结果不是另一个向量，而是一个**标量分数**。后面模型会根据这些分数决定“当前 Token 应该多关注哪些 Token”。

本讲必须真正掌握的一句话

点积 (Dot Product) 就是：**两个等长向量，在相同位置上的数字先相乘，再把所有乘积加起来。**

与 LLM / Transformer / Agent Systems 的连接

你在之前 LLM 课程里已经见过 Q 、 K 、 V 和 Attention，但当时重点是整体结构。现在我们从最底层回答：为什么 Q 和 K 可以“比较”？比较后为什么会得到一个数字？这个数字又受到哪些因素影响？

1.1 本讲学完的最低验收标准

你应该能够：

1. 解释一个向量在深度学习里可以代表什么；
2. 手算 $[1, 2, 3] \cdot [4, 5, 6]$ ；
3. 解释点积的输入 Shape 和输出 Shape；
4. 解释同方向、垂直、反方向时点积符号为什么不同；
5. 区分 Dot Product 与 Cosine Similarity；
6. 说明“点积大”为什么不一定只意味着“方向更相似”；
7. 看懂一个 Query 对多个 Key 的点积打分；
8. 为第 04 讲矩阵乘法和后续 Attention 做好准备。

2 向量到底是什么？

2.1 【原理】向量首先是一串有顺序的数字

最简单的向量可以写成：

$$x = [x_1, x_2, x_3]$$

例如：

$$x = [2, 5, -1]$$

它有 3 个分量，因此可以说它是一个 3 维向量。这里“3 维”不是说 Tensor 有 3 个 axis，而是说这个向量包含 3 个坐标/特征。

Dimension 的两个常见语境

“dimension”在深度学习里经常有两种含义：

- Tensor 的维数 (ndim)：例如 Shape (2, 3, 4) 是 3D Tensor；
- 向量的特征维度：例如 Shape (768,) 的向量常说 hidden dimension = 768。

一定要结合上下文判断。

2.2 【通俗解释】向量就是“一条对象的数字身份证”

现实世界的对象不能直接送进神经网络。我们往往把它表示成一串数字。

例如，用 3 个特征描述一名学生：

[身高, 体重, 年龄]

可以得到：

[175, 70, 20]

在 LLM 中，一个 Token 也会被表示成一个高维向量，例如 hidden size = 4096 时，一个 Token 在某层可能对应 4096 个数。

一个极小的 Token 向量例子

为了方便手算，假设“cat”被表示成：

[0.2, -0.5, 0.8]

真实 LLM 不会只用 3 个数，但数学原理完全一样。

3 Dot Product：对应位置相乘，再求和

3.1 【必要公式】

对两个长度相同的向量：

$$a = [a_1, a_2, \dots, a_n], \quad b = [b_1, b_2, \dots, b_n]$$

点积定义为：

$$a \cdot b = \sum_{i=1}^n a_i b_i$$

展开就是：

$$a \cdot b = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

3.2 【公式逐项拆解】

- a_i ：向量 a 第 i 个位置的数字；
- b_i ：向量 b 第 i 个位置的数字；
- $a_i b_i$ ：同一位置两个数字相乘；
- $\sum$ ：把所有位置得到的乘积加起来；
- 输出：一个 Scalar，而不是一个 Vector。

Shape 规则

若 $a.shape = (D,)$ 且 $b.shape = (D,)$ ，则 $a \cdot b$ 输出一个标量，Shape 可看作 $()$ 。两个向量必须在要配对的特征维度上长度一致。

4 【具体数字例子】完整手算一次

设：

$$a = [1, 2, 3], \quad b = [4, 5, 6]$$

第一步，对应位置相乘：

$$1 \times 4 = 4$$

$$2 \times 5 = 10$$

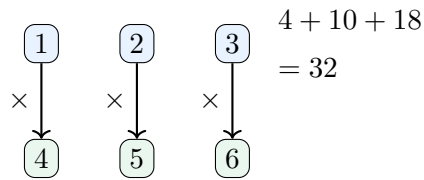
$$3 \times 6 = 18$$

第二步，把三个乘积加起来：

$$4 + 10 + 18 = 32$$

所以：

$$a \cdot b = 32$$



常见误区 / 风险

点积不是逐元素乘法本身。逐元素乘法 $a * b$ 的结果还是向量 $[4, 10, 18]$ ；点积还要再求和，所以结果是一个数 32。

5 为什么点积可以反映“方向关系”？

5.1 从最简单的二维向量开始

设：

$$q = [1, 0]$$

考虑三个 Key：

$$k_1 = [2, 0], \quad k_2 = [0, 3], \quad k_3 = [-2, 0]$$

分别计算：

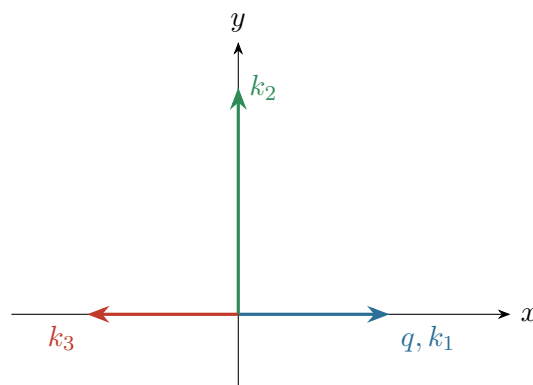
$$q \cdot k_1 = 1 \times 2 + 0 \times 0 = 2$$

$$q \cdot k_2 = 1 \times 0 + 0 \times 3 = 0$$

$$q \cdot k_3 = 1 \times (-2) + 0 \times 0 = -2$$

从几何上看：

- q 与 k_1 同方向，点积为正；
- q 与 k_2 垂直，点积为 0；
- q 与 k_3 反方向，点积为负。



5.2 更完整的几何公式

点积也可以写成：

$$a \cdot b = \|a\| \|b\| \cos \theta$$

其中：

- $\|a\|$ ：向量 a 的长度；
- $\|b\|$ ：向量 b 的长度；
- θ ：两个向量之间的夹角；
- $\cos \theta$ ：描述方向关系。

这里不需要背证明

当前阶段只需要建立直觉：点积同时包含两部分信息：**方向是否一致**，以及**两个向量本身有多长**。

6 “点积大 = 更相似” 为什么只是近似说法？

很多入门材料会直接说：“点积越大，两个向量越相似。”这句话有用，但不够严谨。

看一个例子：

$$x = [1, 1]$$

$$y_1 = [1, 1], \quad y_2 = [10, 10]$$

显然 y_1 和 y_2 与 x 的方向完全相同。

但是：

$$x \cdot y_1 = 2$$

$$x \cdot y_2 = 20$$

第二个点积大 10 倍，并不是因为方向“更像”，而是因为 y_2 更长。

重要结论

Dot Product 同时受到**方向**和**长度 (magnitude)**影响。因此在严格讨论“纯方向相似度”时，经常使用 Cosine Similarity。

7 Cosine Similarity：把长度影响除掉

7.1 【必要公式】

$$\cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$$

向量长度 (L2 norm) 为:

$$\|a\| = \sqrt{a_1^2 + a_2^2 + \dots + a_n^2}$$

7.2 为什么这样做?

分子是点积, 包含“方向 + 长度”; 分母用两个向量长度做归一化, 相当于把尺度影响除掉, 留下主要的方向关系。

对于非零向量, Cosine Similarity 常在 $[-1, 1]$ 范围内:

- 接近 1: 方向接近;
- 接近 0: 接近垂直;
- 接近 -1: 方向相反。

不要把 Dot Product 和 Cosine Similarity 混为一谈

它们相关, 但不是同一个量。Transformer Attention 的标准打分使用的是 **scaled dot product**, 不是直接使用 cosine similarity。

8 为什么神经网络喜欢点积?

8.1 原因 1: 它把两个向量压缩成一个匹配分数

很多任务需要回答: “向量 A 与向量 B 匹配得怎么样?” 点积恰好输出一个标量。

8.2 原因 2: 计算极其规则, 适合矩阵化和 GPU

如果有很多 Query 和 Key, 不需要写 Python 循环逐个比较。可以把向量堆成矩阵, 一次矩阵乘法完成大量点积。这就是下一讲矩阵乘法的重要动机。

8.3 原因 3: 它可微分

未来学习 Backpropagation 后会看到: 点积由乘法和加法组成, 梯度容易计算, 因此可以端到端训练。

科研视角

当论文把“相似度函数”从 dot product 换成 cosine、bilinear score、learned score 时, 本质上是在改变“什么叫匹配”的数学定义。Ablation 往往就是在检验这种设计是否真的重要。

9 从一个 Query 到多个 Key: 第一次接近 Attention

假设一个 Query:

$$q = [1, 2]$$

有三个 Key:

$$k_1 = [1, 0], \quad k_2 = [0, 1], \quad k_3 = [1, 1]$$

逐个点积:

$$q \cdot k_1 = 1$$

$$q \cdot k_2 = 2$$

$$q \cdot k_3 = 3$$

得到分数:

$$[1, 2, 3]$$

与 Attention 的直接连接

以后 Attention 会做类似事情: 当前 Token 产生 Query, 所有候选 Token 产生 Key; Query 与每个 Key 做点积, 得到一排 score。随后还要经历缩放、Mask、Softmax, 才得到最终注意力权重。本讲只负责把最底层的“打分”机制学透。

10 为什么 Attention 还要除以 $\sqrt{d_k}$? 先埋一个伏笔

标准 Scaled Dot-Product Attention 中有:

$$\frac{QK^T}{\sqrt{d_k}}$$

为什么要除以 $\sqrt{d_k}$?

现在只需要知道: 当向量维度 d_k 很大时, 很多乘积相加后, 点积的绝对值容易变大。数值过大送入 Softmax, 会使概率分布过于尖锐, 梯度也可能变得不理想。

后面学完概率、Softmax、矩阵乘法后, 我们会重新严格解释这一点。

学习策略

这里先建立“高维点积的尺度会变大, 所以需要 scale”的直觉, 不要求现在推导方差公式。Just-in-Time Math 的原则就是: 用到哪里, 补到哪里。

11 【代码实验】先手工逻辑, 再 NumPy

最小代码:

```
import numpy as np

a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])

products = a * b
print(products)           # [4. 10. 18.]
```

```
print(products.sum())    # 32.0
print(np.dot(a, b))      # 32.0
```

这三行形成非常重要的对应：

$a * b \rightarrow$ 逐元素乘法

$\text{sum} \rightarrow$ 求和

$\text{np.dot} \rightarrow$ 把两步封装成点积

11.1 向量长度与余弦相似度

```
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)
cosine = np.dot(a, b) / (norm_a * norm_b)
```

本讲完整实验代码还会比较：

1. 同方向；
2. 垂直；
3. 反方向；
4. 同方向但长度不同；
5. 一个 Query 对多个 Key；
6. 可选 PyTorch 结果核对。

12 Shape 视角：不要只盯着数字

若：

$$q.shape = (D,)$$

$$k.shape = (D,)$$

则：

$$q \cdot k \rightarrow scalar$$

若把 3 个 Key 堆起来：

$$K.shape = (3, D)$$

那么一次计算：

$$Kq$$

就能得到：

$$score.shape = (3,)$$

即 3 个 Key 各自对应一个分数。

从点积自然走向矩阵乘法

“很多次点积”本质上可以打包成一次矩阵乘法。第 04 讲就是把这个思想系统化。

13 常见误区

误区 1：向量就是箭头

箭头是非常有用的几何解释，但深度学习里的高维向量通常无法直接画出来。最根本的理解仍然是“一组有顺序的数字”。

误区 2：点积一定在 $[-1, 1]$

错误。Cosine Similarity 常在 $[-1, 1]$ ，普通 Dot Product 没有这个限制。例如 $[10, 10] \cdot [10, 10] = 200$ 。

误区 3：点积越大就一定语义越相似

点积只是数学打分。一个训练好的模型可能学习到有意义的向量空间，但不能脱离模型、训练目标和向量尺度，把任何点积都自动解释为“语义相似度”。

误区 4：两个不同长度的向量也能直接点积

标准点积要求对应维度能够一一配对。例如 Shape $(3,)$ 与 $(4,)$ 无法直接做普通向量点积。

14 科研视角：以后论文中你会怎样遇到它？

常见写法包括：

- similarity score;
- dot-product attention;
- cosine similarity;
- query-key matching;
- representation similarity;
- embedding similarity;
- normalized embeddings。

看到这些词时，至少要追问：

1. 比较的两个向量分别代表什么？

- 2. Shape 是什么？
- 3. 使用 dot product 还是 cosine？
- 4. 是否做了 normalization？
- 5. score 后面是否经过 Softmax？
- 6. 改变打分函数会不会影响实验结论？

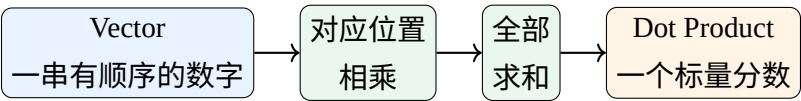
Ablation 的一个典型问题

如果一个新 Attention 方法声称“新的 similarity function 更好”，审稿人通常会关心：提升来自打分函数本身，还是来自参数量、归一化、训练配置等其他变化？因此研究里经常需要控制变量进行 ablation。

15 把本讲重新接回你已经学过的 LLM 概念

以前见过的概念	现在补上的底层理解
Embedding	一个 Token 最终会被表示成高维向量
Q / K	都是由 hidden state 经过线性变换得到的向量
Attention Score	Query 与 Key 的点积形成基础匹配分数
Scaled Dot Product	点积还会按维度进行缩放
Softmax	后面把 score 转成归一化权重
Multi-Head Attention	多组 Q/K/V 在不同子空间分别做类似计算

16 本讲知识链



再向后连接：

Dot Product → Matrix Multiplication → QK^T → Attention

17 本讲自检

- 不看答案，尝试回答：
- 1. $[2, -1, 3] \cdot [4, 5, 2]$ 等于多少？
 - 2. 点积输出为什么是一个 Scalar？
 - 3. 为什么两个垂直向量的点积是 0？

4. 为什么 $[1, 1] \cdot [10, 10]$ 比 $[1, 1] \cdot [1, 1]$ 大很多，但不能说方向更相似？
5. Dot Product 和 Cosine Similarity 的关键区别是什么？
6. Attention 中 $Q \cdot K$ 的作用是什么？
7. 为什么多个 Key 可以堆成矩阵，一次得到多个 score？

本节检查点

如果你能手算点积、预测输入输出 Shape、解释“方向 + 长度”共同影响点积，并能把它连接到 Query-Key 打分，本讲就已经达到要求。不要追求几何证明。

18 下一讲预告：Matrix Multiplication

下一讲将解决：

如果一次要对几十、几百、几千个向量做点积，难道一个个算吗？

答案是：把很多点积组织成**矩阵乘法**。

我们将手算：

$$A_{2 \times 3} B_{3 \times 2}$$

并最终连接：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

这会是整个深度学习课程最关键的数学基础课之一。